The Character Streams and Multithreaded Programming

```java
1. import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.io.Reader;
import java.io.Writer;
import java.io.IOException;

public class KeyboardToConsole {
    public static void main(String[] args) {
        System.out.println("Enter text (Press Ctrl+D or Ctrl+Z to stop):");
        try (Reader reader = new InputStreamReader(System.in);
             Writer writer = new PrintWriter(System.out)) {

            int character;
            while ((character = reader.read()) != -1) {
                writer.write(character);
                writer.flush();
            }
        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
        }
    }
}
```

Output:
C:\25EU02121>javac KeyboardToConsole.java
C:\25EU02121>java KeyboardToConsole
Enter text (Press Ctrl+D or Ctrl+Z to stop):
Java
Java
Programming
Programming
^Z

```java
2. import java.io.FileReader;
import java.io.IOException;

public class ReadFileDemo {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("input.txt")) {
            int character;
            System.out.println("--- Content of input.txt ---");
            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
```

```
            }
            System.out.println("\n----------------------------");
        } catch (IOException e) {
            System.err.println("Error reading the file: " + e.getMessage());
        }
    }
}
```

Output:
C:\25EU02121>javac ReadFileDemo.java
C:\25EU02121>java ReadFileDemo
Error reading the file: input.txt (The system cannot find the file specified)

```
3. import java.io.FileWriter;
import java.io.IOException;

public class WriteFileDemo {
    public static void main(String[] args) {
        String textToWrite = "Hello! This text is written using Java
FileWriter.\nWelcome to Character Streams.";

        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write(textToWrite);
            System.out.println("Successfully wrote data to output.txt");
        } catch (IOException e) {
            System.err.println("Error writing to the file: " + e.getMessage());
        }
    }
}
```

Output:
C:\25EU02121>javac WriteFileDemo.java
C:\25EU02121>java WriteFileDemo
Successfully wrote data to output.txt

```
4. import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopyFileDemo {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("source.txt");
             FileWriter writer = new FileWriter("destination.txt")) {

            int character;
```

```
            while ((character = reader.read()) != -1) {
                writer.write(character);
            }
            System.out.println("File copied successfully from source.txt to
destination.txt");
        } catch (IOException e) {
            System.err.println("Error during file copy operation: " +
e.getMessage());
        }
    }
}
```

Output:
C:\25EU02121>javac CopyFileDemo.java
C:\25EU02121>java CopyFileDemo
Error during file copy operation: source.txt (The system cannot find the file specified)

```
5. import java.io.FileReader;
import java.io.IOException;

public class FileCounterDemo {
    public static void main(String[] args) {
        int charCount = 0;
        int wordCount = 0;
        int lineCount = 0;
        boolean inWord = false;
        try (FileReader reader = new FileReader("input.txt")) {
            int character;
            while ((character = reader.read()) != -1) {
                charCount++;
                char ch = (char) character;
                if (ch == '\n') {
                    lineCount++;
                }
                if (Character.isWhitespace(ch)) {
                    inWord = false;
                } else if (!inWord) {
                    wordCount++;
                    inWord = true;
                }
            }
            if (charCount > 0 && lineCount == 0) {
                lineCount = 1;
            }
            System.out.println("Characters: " + charCount);
```

```
            System.out.println("Words: " + wordCount);
            System.out.println("Lines: " + lineCount);
        } catch (IOException e) {
            System.err.println("Error analyzing file: " + e.getMessage());
        }
    }
}
```

Output:
C:\25EU02121>javac FileCounterDemo.java
C:\25EU02121>java FileCounterDemo
Characters: 23
Words: 4
Lines: 2

(`input.txt` contains exactly two lines of text: "Hello Java\nStreams Rule")

6.
```
public class MainThreadDemo {
    public static void main(String[] args) {
        Thread mainThread = Thread.currentThread();

        System.out.println("Main Thread Information:");
        System.out.println("Name: " + mainThread.getName());
        System.out.println("Priority (1-10): " + mainThread.getPriority());
        System.out.println("State: " + mainThread.getState());
    }
}
```

Output:
C:\25EU02121>javac MainThreadDemo.java
C:\25EU02121>java MainThreadDemo
Main Thread Information:
Name: main
Priority (1-10): 5
State: RUNNABLE

7.
```
class CustomThread extends Thread {
    @Override
    public void run() {
        System.out.println(getName() + " is running (Extended from Thread
class).");
    }
}
public class ThreadExtensionDemo {
    public static void main(String[] args) {
        CustomThread thread = new CustomThread();
```

```
        thread.setName("WorkerThread-1");
        thread.start();
    }
}
```

Output:
C:\25EU02121>javac ThreadExtensionDemo.java
C:\25EU02121>java ThreadExtensionDemo
WorkerThread-1 is running (Extended from Thread class).

```
8. class CustomRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " is running
(Implemented via Runnable).");
    }
}

public class RunnableThreadDemo {
    public static void main(String[] args) {
        CustomRunnable runnable = new CustomRunnable();
        Thread thread = new Thread(runnable, "WorkerThread-2");
        thread.start();
    }
}
```

Output:
C:\25EU02121>javac RunnableThreadDemo.java
C:\25EU02121>java RunnableThreadDemo
WorkerThread-2 is running (Implemented via Runnable).

```
9. class InterleavedTask implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " processing
item: " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println("Thread execution interrupted");
            }
        }
    }
}
```

```
public class ConcurrentExecution {
    public static void main(String[] args) {
        Thread t1 = new Thread(new InterleavedTask(), "Thread-A");
        Thread t2 = new Thread(new InterleavedTask(), "Thread-B");
        Thread t3 = new Thread(new InterleavedTask(), "Thread-C");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output:
C:\25EU02121>javac ConcurrentExecution.java
C:\25EU02121>java ConcurrentExecution
Thread-A processing item: 1
Thread-B processing item: 1
Thread-C processing item: 1
Thread-A processing item: 2
Thread-C processing item: 2
Thread-B processing item: 2
Thread-A processing item: 3
Thread-B processing item: 3
Thread-C processing item: 3

10.
```
public class IsAliveDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread thread = new Thread(() -> {
            try { Thread.sleep(300); } catch (InterruptedException ignored) {}
        });

        System.out.println("Before start() - Is thread alive? " +
thread.isAlive());

        thread.start();
        System.out.println("After start() - Is thread alive? " +
thread.isAlive());

        thread.join();
        System.out.println("After completion - Is thread alive? " +
thread.isAlive());
    }
}
```

Output:
C:\25EU02121>javac IsAliveDemo.java
C:\25EU02121>java IsAliveDemo
Before start() - Is thread alive? false
After start() - Is thread alive? true
After completion - Is thread alive? False

11.
```
public class JoinMethod {
    public static void main(String[] args) {
        Thread dependentThread = new Thread(() -> {
            System.out.println("Dependent thread starting heavy
computations...");
            try {
                Thread.sleep(1000);
                System.out.println("Dependent thread work finished.");
            } catch (InterruptedException e) {
                System.err.println(e.getMessage());
            }
        });

        dependentThread.start();

        System.out.println("Main thread waiting for dependent thread to conclude
via join()...");
        try {
            dependentThread.join();
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }

        System.out.println("Main thread execution resumes and wraps up.");
    }
}
```

Output:
C:\25EU02121>javac JoinMethod.java
C:\25EU02121>java JoinMethod
Main thread waiting for dependent thread to conclude via join()...
Dependent thread starting heavy computations...
Dependent thread work finished.
Main thread execution resumes and wraps up.

12.
```
public class ControlledExecution {
    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            System.out.println(Thread.currentThread().getName() + " has begun.");
```

```
            try { Thread.sleep(400); } catch (InterruptedException ignored) {}
            System.out.println(Thread.currentThread().getName() + " has
completed.");
        };

        Thread t1 = new Thread(task, "Stage-1");
        Thread t2 = new Thread(task, "Stage-2");
        Thread t3 = new Thread(task, "Stage-3");

        t1.start();
        System.out.println("Checking Stage-1 running status: " + t1.isAlive());
        t1.join();

        t2.start();
        t2.join();

        t3.start();
        t3.join();

        System.out.println("All controlled sequencing stages have completed
successfully.");
    }
}
```

Output:
C:\25EU02121>javac ControlledExecution.java
C:\25EU02121>java ControlledExecution
Checking Stage-1 running status: true
Stage-1 has begun.
Stage-1 has completed.
Stage-2 has begun.
Stage-2 has completed.
Stage-3 has begun.
Stage-3 has completed.
All controlled sequencing stages have completed successfully.